SCP 2026 第二轮（复赛 S 组）模拟

两题合订题解 · P17722 档案（file） · P17723 函数（func）

题面与数据范围 → 样例图示手算 → 好拿的部分分阶梯
→ 穿插式正解推导 → 实测通过的满分代码 → 对拍验证方法

适合正在学习贪心与位运算的同学，从 0 基础一路读到满分

目录

第 I 部分 · P17722 档案（file）	第 II 部分 · P17723 函数（func）
1 题面与数据范围	1 题面与数据范围
2 样例手算（看图理解）	2 样例手算
3 部分分路线地图	3 部分分路线地图
档 1 枚举所有顺序（10 分）	档 1 $O(n^3)$ 暴力（10 分）
档 2 状压 DP（25 分）	档 2 $O(n^2)$ 增量维护（30 分）
4 正解推导（用到什么推什么）	4 正解推导：MEX 的灯早被 OR 点亮
档 3 满分代码（100 分）	档 3 满分代码（100 分）
5 特殊性质点 13、14 怎么顺手拿	5 func 题常见坑
6 file 题常见坑	6 附录：如何用对拍验证代码

怎么用这份题解

1. 每题先读「题面」和「样例手算」，把题意彻底搞清楚，再动手写代码；
2. 再按「部分分路线地图」从档 1 往后写——每档都是一份能独立得分的完整程序，一档比一档多走一步；
3. 所有「为什么这样做」的道理都放在「正解推导」一节，用到什么推什么，不扎堆；
4. 写完代码后，读「常见坑」和文末「对拍」附录，自己动手验证一遍。

不认识的词先别慌 文中会反复用到几个小工具，这里先打个照面：
· **排序**：把一串数从小到大排好。 · **二分查找 / lower_bound**：在**已排序**的数组里快速定位某个值的位置，`lower_bound` 返回「第一个 $\geqslant x$ 的元素」的位置。 · **按位或 OR**：两个数对应二进制位「有一个是 1 结果就是 1」。 · **MEX**：一堆数里「从 0 开始第一个没出现的数」，下文第 II 部分会专门讲。

第 I 部分 · P17722 档案（file）

1. 题面与数据范围

题意：有 n 个核心档案（序列 a）已归档，另有 m 批待归档档案，第 i 批是序列 b_i（长 l_i）。所有编号两两不同。 每次操作选一批还没归档的档案加入当前序列 x 并从小到大排序，**本次代价 = 排序后所有核心档案的下标之和**。 m 次操作的顺序由你决定，求总代价最小值。

> **「下标之和」是什么？** 下标就是**位置编号，从 1 开始数**。比如序列 [1,3,5] 里，核心档案 3 在第 2 个位置，它的下标就是 2。 一次操作的代价，就是把每个核心档案此刻所在的位置编号全部加起来。我们要做的，是安排 m 批档案的**加入顺序**，让 m 次操作的代价加起来最小。

输入：第一行 n m；第二行 n 个核心编号；接下来 m 行，每行 l_i 加 l_i 个编号。

输出：一行，最小总代价。

数据范围：设 $L = \Sigma l_i$。所有测试点满足 n, m, L $\leqslant 10^6$，编号 $\leqslant 2\times10^6$，全部编号互不相同。

测试点	m ⩽	n, L ⩽	特殊性质	分值占比
1~2	7	100	无	10 分
3~5	11	1000	无	15 分
6~8	100	10^4	无	15 分
9~12	10^5	10^6	无	20 分
13	10^6	10^6	A：n = 1	5 分
14	10^6	10^6	B：核心连续、每批连续	5 分
15~20	10^6	10^6	无	30 分

2. 样例手算（看图理解）

样例 1：核心 a = [2, 5]，两批 [1,3]、[4,6]。一共只有 2 批，顺序只有两种，我们把**两种都老老实实算一遍**，看看哪种更优（深蓝色块是核心档案）：

样例 1：两种顺序对比

顺序 A：先 [1,3] 后 [4,6] → 总代价 13

第 1 次后 1 **2** 3 **5** 位置 2+4 = 6



顺序 B：先 [4,6] 后 [1,3] → 总代价 11 √ 最优



> **想一想：**两种顺序最后得到的序列其实**一模一样**（都是 [1,2,3,4,5,6]），为什么总代价不同？ 差别出在**过程**：顺序 A 在第 1 次就把 1、3 加进来，把核心 2 和 5 早早挤到了后面；顺序 B 先加 4、6（它们只在右边，不挤核心），第 1 次代价只有 4。 这提示我们：**会把核心往后挤的批次，越晚加入越好**。先记住这个直觉，第 4 节会把它变成严格的结论。

样例 2：核心 a = [7,10,15,19,25]，四批 [1]、[16,21,20]、[12,17,29]、[5,9,11,18,24,26]。最优顺序是 **2, 1, 3, 4**，四步代价分别为 19、24、29、44，总代价 **116**。我们一步步验证（浅蓝色块是当步新加入的批次元素）：

样例 2：四步完整验算



合计 19 + 24 + 29 + 44 = 116

3. 部分分路线地图

下面按「能拿多少分」把做法分成三档。**建议一档一档写**：每档都是完整可交的代码，先把低分稳稳拿到，再逐步冲击满分。

档位	做法	复杂度	拿哪些点	累计
档 1	枚举所有顺序 + 模拟	O(m! · m · (n+L) log(n+L))	1~2	10 分
档 2	状压 DP（预处理每集合的当趟代价）	O(2^m · (n+L) log(n+L))	1~5	25 分
档 3	贪心 + 闭式公式（满分）	O((n+L) log n)	全部 20 点	100 分

特殊性质点 13（n=1）与 14（连续编号）不需要单独的程序——档 3 满分代码直接覆盖；想快速蹭分见第 5 节。

档 1　枚举所有顺序（点 1~2，10 分）

思路：$m \leqslant 7$ 时，一共最多 7! = 5040 种顺序，干脆**全部试一遍**，每种顺序老实模拟算代价，取最小值。这一档的目的是「先把题读懂、把样例算对」。

```
// 档 1：枚举全排列（对应测试点 1~2，m <= 7，n、L <= 100）
// m 很小，最多 7! = 5040 种顺序，全部试一遍，每种顺序老实模拟
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = (ll)4e18;

int n, m, a[105];
vector<int> b[8];    // b[i] = 第 i 批档案的编号
bool vis[8];         // vis[i] = 第 i 批在当前排列里用过了吗
int ord[8];          // ord[k] = 第 k 个归档的是哪一批
ll ans;              // 目前见过的最小总代价

// 按 ord[] 的顺序把 m 批全归一遍档，返回总代价
ll calc() {
    vector<int> c(a, a + n);
    ll s = 0;
    for (int k = 0; k < m; k++) {
        for (int x : b[ord[k]]) c.push_back(x);
        sort(c.begin(), c.end());
        for (int j = 0; j < n; j++)
            s += lower_bound(c.begin(), c.end(), a[j]) - c.begin() + 1;  // 下标从 1 数起
    }
    return s;
}

void dfs(int d) {    // 生成 0..m-1 的所有排列
    if (d == m) { ans = min(ans, calc()); return; }
    for (int i = 0; i < m; i++) {
        if (vis[i]) continue;
        vis[i] = 1; ord[d] = i;
        dfs(d + 1);
```

```
        vis[i] = 0;
    }
}

int main() {
    scanf("%d %d", &n, &m);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 0; i < m; i++) {
        int l; scanf("%d", &l);
        for (int j = 0; j < l; j++) { int x; scanf("%d", &x); b[i].push_back(x); }
    }
    ans = INF; dfs(0);
    printf("%lld\n", ans);
    return 0;
}
```

档 2　状压 DP（点 1~5，25 分）

思路：m ⩽ 11 时枚举排列有 3991 万种，太慢；但枚举「**哪些批归过了**」只有 $2^{11} = 2048$ 种，够快。这里用一个叫**状态压缩（状压）**的小技巧：用一个二进制数 S 记录集合——**S 的第 i 位是 1 就表示第 i 批已经归档过**，是 0 就表示还没。 于是 dp[S] 表示「按某种顺序归档完集合 S 里这些批，能得到的最小总代价」。每次往集合里加一批，把这一步的代价加上去即可。 这样做的好处是：不同的加入顺序如果最后归档的是同一批集合，会被自动合并成同一个状态，所以比档 1 快得多。

还有一个**能救命的小观察**：往 S 里加第 i 批，这一步付多少代价，只取决于**归完之后**的集合 T = S ∪ {i}——跟顺序无关。 而转移有 2048 × 11 ≈ 2 万条，如果每条转移都重新排序模拟一遍，总计约 10^9 次操作，**必超时**（真实评测里这一档的 3~5 号点就是这样 TLE 的）。 所以正确的写法是：**2048 个集合各预处理一次代价 step[T]**，DP 转移时直接查表，预处理只算 2048 次而不是 2 万次。

```
// 档 2：状压 DP（对应测试点 1~5，m <= 11，n、L <= 1000）
// 枚举"哪些批归过了"只有 2^11 = 2048 种，比枚举排列快得多
// 关键观察：归完集合 T 后这一趟的代价只和 T 有关，与顺序无关，
// 所以 2048 个集合各预处理一次，DP 转移时直接查表
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const ll INF = (ll)4e18;

int n, m, a[1005];
vector<int> b[12];
ll step[1 << 11];    // step[T] = 归完集合 T 后，这一趟要付的代价
ll dp[1 << 11];      // dp[S] = 归完集合 S 里这些批的最小总代价

int main() {
    scanf("%d %d", &n, &m);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 0; i < m; i++) {
        int l; scanf("%d", &l);
        for (int j = 0; j < l; j++) { int x; scanf("%d", &x); b[i].push_back(x); }
    }
    int N = 1 << m;
    // 预处理每个集合的当趟代价：把核心和 T 里的元素排一起，数核心位置
    for (int T = 1; T < N; T++) {
        vector<int> c(a, a + n);
        for (int k = 0; k < m; k++) if (T >> k & 1)
            for (int x : b[k]) c.push_back(x);
        sort(c.begin(), c.end());
        ll s = 0;
        for (int j = 0; j < n; j++)
            s += lower_bound(c.begin(), c.end(), a[j]) - c.begin() + 1;
        step[T] = s;
    }
    for (int S = 1; S < N; S++) dp[S] = INF;
    dp[0] = 0;                          // 空集：什么都没归，代价 0
    for (int S = 0; S < N; S++)
        for (int i = 0; i < m; i++) {
            if (S >> i & 1) continue;   // 这批已经归过了
            int T = S | (1 << i);       // 把第 i 批加进集合
```

```
            dp[T] = min(dp[T], dp[S] + step[T]);
        }
    printf("%lld\n", dp[N - 1]);        // 全部归完的最小总代价
    return 0;
}
```

> **这一档慢在哪？** 注意预处理 step[T] 时还是要把整条序列**重新排序**一遍，2048 个集合各排一次已经是这档的极限。 这正是我们想继续往下走的原因：有没有办法**不用真的去排序模拟**，就能算出每一步的代价？答案就在下一节。

4. 正解推导（用到什么推什么）

这一节回答三个问题，每个问题都对应一个观察。读完你会发现：原来不用模拟，直接用公式就能算出答案。

4.1 第一个观察：顺序只影响「非核心元素」

每次操作后，核心档案 a_j 的下标 = 序列里「$\leqslant a_j$ 的元素个数」。这些元素分两类：**核心自己**（从头到尾都在序列里，贡献固定，跟你安排的顺序无关）和**待归档元素**（什么时候进来，由你决定）。所以，**真正值得优化的只有第二类**。

4.2 第二个观察：一个元素的代价 = 破坏力 × 被计入次数

元素 x 能「挤动」的核心档案，只有**比它大的那些**（比它小的核心永远排在它前面，不受影响；编号互不相同，不存在相等）。于是定义 x 的**破坏力**：

$$w(x) = |\{\, j : a_j > x \,\}| \quad \text{（比 x 大的核心个数）}$$

若 x 在第 t 次操作被归档，它会出现在第 t, t+1, ..., m 次的序列里，一共被计入 `m - t + 1` 次，所以它的总贡献 = $w(x) \times (m - t + 1)$。 一批档案同时进来，把批内所有元素的 w 加起来，就是这一批的**批次破坏力**：

$$W_i = \Sigma_{x \in b_i} w(x)$$

4.3 第三个观察：系数固定，配对用排序不等式

第 1 个处理的批次被计入 m 次，第 2 个是 m−1 次，……最后一个是 1 次。而核心自己的固定贡献是

$$C_0 = \Sigma_{j=1..n} |\{\, k : a_k \leqslant a_j \,\}| = 1+2+...+n = n(n+1)/2$$

（因为编号互不相同，排序后第 j 个核心前面恰好有 j 个核心）。于是总代价可以写成：

$$Ans = m \cdot C_0 + \Sigma_{i=1..m} W_i \cdot (m - t_i + 1)$$

剩下的问题是：怎么把各批的 W 和系数 m, m−1, ..., 1 配对，让乘积之和最小？看相邻两个位置（系数分别是 k 和 k−1，放着 p、q 两批），比较「交换前」和「交换后」的差：

$$\Delta = （交换前）-（交换后）= (W_p \cdot k + W_q \cdot (k-1)) - (W_q \cdot k + W_p \cdot (k-1)) = W_p - W_q$$

当 $W_q < W_p$ 时 $\Delta > 0$，说明交换前更大、交换后更小——**交换更划算，也就是小的 W 应该放前面**。不断做这样的交换，最终得到：**按 W 从小到大排列最优**（这就是排序不等式的思想）。W 相同的批次谁先谁后都一样。

> **正解一句话：**给每个待归档元素算 w(x) = 比它大的核心个数（把核心排序后二分查找），加成批次 W；按 W 从小到大归档；答案 = $m \cdot C_0 + \Sigma W_i \cdot (m-序号+1)$。复杂度 O((n+L) log n)。

档 3　满分代码 O((n+L) log n)

把上面三条结论原样翻译成代码即可：

```
// 档 3：满分代码 O((n+L) log n)（对应全部 20 个测试点）
// 三条结论：
//   1) 核心自己的贡献是常数 C0 = 1+2+...+n = n(n+1)/2
//   2) 元素 x 的破坏力 w(x) = 比 x 大的核心个数，批内求和得 W
//   3) 按 W 从小到大归档，答案 = m*C0 + 第 i 批的 W * (m-i+1)
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    int n, m;
    scanf("%d %d", &n, &m);
    vector<int> a(n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    sort(a.begin(), a.end());
    ll c0 = (ll)n * (n + 1) / 2;   // 核心自己的固定贡献
    ll ans = (ll)m * c0;           // m 次操作每次都付一份
    vector<ll> w(m);               // w[i] = 第 i 批的破坏力
    for (int i = 0; i < m; i++) {
        int l; scanf("%d", &l);
        ll s = 0;
        for (int j = 0; j < l; j++) {
            int x; scanf("%d", &x);
            // upper_bound 给出 <= x 的核心个数，用 n 减掉就是 > x 的个数
            s += n - (int)(upper_bound(a.begin(), a.end(), x) - a.begin());
        }
        w[i] = s;
    }
    sort(w.begin(), w.end());      // 破坏力小的先归档，系数最大为 m
    for (int i = 0; i < m; i++) ans += w[i] * (m - i);
    printf("%lld\n", ans);
    return 0;
}
```

5. 特殊性质点 13、14 怎么顺手拿（不用新程序）

这两个点（共 10 分）档 3 的满分代码直接就能过，因为通用做法本来就涵盖它们。如果考场上还没推出正解，只想快速蹭分，就在档 3 代码的基础上做两处「省掉二分」的简化：

- **点 13（$n = 1$）：** 核心只有一个数 A，所以 w(x) 非 0 即 1——x < A 就是 1，否则 0。于是 W_i = 批内比 A 小的元素个数，一个循环数完；常数 $C_0 = 1$。
- **点 14（编号连续）：** 核心是连续一段 A, A+1, ..., A+n−1，w(x) 可以口算：x < A → n；A ⩽ x < A+n−1 → A+n−1−x；否则 0。

两处都只需把「二分求 w(x)」那一行换成口算，其余代码一字不改。

6. file 题常见坑

1. 下标从 1 开始，lower_bound 给出的是从 0 数的位置，记得 +1。
2. 答案开 long long：$m \cdot C_0$ 一项就可达 $10^6 \times 5 \times 10^{11} \approx 5 \times 10^{17}$，爆 int。
3. 数「比 x 大」用 upper_bound（它给的是 ⩽ x 的个数），用 n 减掉才是严格大于。
4. 核心数组要先排序再二分。
5. 系数方向：第 1 个处理的批次系数是 m（最大），不是 1。
6. n = 1 特化和连续特化只是少写二分，通用代码（档 3）也能过这两个点——时间紧时不必单独写。
7. C_0 不用循环算：编号互不相同，核心自己的贡献恒为 1+2+⋯+n，直接 n*(n+1)/2 即可（注意先转 long long 再乘）。
8. 状压 DP 千万别在每条转移里重新模拟：转移有 2^m × m 条，各模拟一次排序总计约 10^9 次操作，3~5 号点必 TLE。当趟代价只跟「归完后的集合」有关，2^m 个集合各预处理一次即可。

第 II 部分 · P17723 函数（func）

1. 题面与数据范围

题意：给长度 n（$\leqslant 2\times10^6$）的非负整数序列 a（$a_i < 2^{30}$）。对区间 [l, r]，把里面出现的数装进集合 S（重复的只算一个），定义两个函数：

- **OR(S)**：所有元素按位或——大白话：「把所有人的灯合并点亮」；
- **MEX(S)**：不在 S 中的最小非负整数——大白话：「从 0 开始点名，第一个没人答到的数」。

区间价值 f(l,r) = OR(S) ∨ MEX(S)。求所有 n(n+1)/2 个区间的价值之和，**对 2^{64} 取模**。输入量大，注意用快速读入。

> **对 2^{64} 取模的最省事写法：**全程用 `unsigned long long`，让它自然溢出就等于自动取模，最后 `%llu` 输出。**不要**用有符号 long long 溢出（那是未定义行为，结果不可预料）。

测试点	n ⩽	$a_i <$	特殊性质（随机生成）	分值占比
1	10	4	×	5 分
2	100	2^{10}	×	5 分
3	1000	2^{20}	×	5 分
4	3000	2^{20}	×	5 分
5~6	5000	2^{30}	×	10 分
7~8	$2\times10^4 \sim 5\times10^4$	2^{30}	✓ 随机	10 分
9~14	$10^5 \sim 5\times10^5$	$2^{10} \sim 2^{30}$	×	30 分
15~16	$10^6 \sim 1.5\times10^6$	$2^{10} \sim 2^{20}$	✓ 随机	10 分
17~20	2×10^6	2^{30}	×	20 分

2. 样例手算

样例 1：a = [0, 1, 2]，共 6 个区间，逐个算一遍：

区间	S	OR	MEX	f = OR ∨ MEX
[1,1]	{0}	0	1	1
[2,2]	{1}	1	0	1

[3,3]	{2}	2	0	2
[1,2]	{0,1}	1	2	3
[2,3]	{1,2}	3	0	3
[1,3]	{0,1,2}	3	3	3
			合计	**13**

样例 2：a = [1,0,1,3] → **24**（10 个区间的 f 依次为 1,1,1,3,3,3,3,3,3,3）。样例 3：a = [536870912,0,1,2,3,536870912,1073741823,0,0,1] → **35970351152**。

3. 部分分路线地图

档位	做法	复杂度	拿哪些点	累计
档 1	三重循环暴力	$O(n^3)$	1~2	10 分
档 2	固定左端点，增量维护 OR/MEX	$O(n^2)$	1~6	30 分
档 3	满分：OR 按位统计 + 修正项切段计数	O(n log n)	全部	100 分

点 7、8、15、16 的「随机生成」性质不需要单独的程序——档 3 满分做法本来就能过所有点；它的核心想法（按位统计）比任何针对随机性质的投机做法都更直白，建议直接奔满分。

档 1　$O(n^3)$ 暴力（点 1~2，10 分）

思路：把每一个区间都真的算一遍 OR 和 MEX。目的是先保证自己完全读懂了题意。

```
// 档 1：最直白的三重循环暴力（对应测试点 1~2，n <= 100）
// 把每一个区间都真的算一遍，先保证自己读懂题意
#include <bits/stdc++.h>
using namespace std;
typedef unsigned long long ull;   // ull 自然溢出就等于"对 2^64 取模"

int n;
unsigned a[105];

int main() {
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) scanf("%u", &a[i]);
    ull ans = 0;
    for (int l = 1; l <= n; l++)            // 枚举左端点
        for (int r = l; r <= n; r++) {      // 枚举右端点
            bool vis[105] = {0};            // 桶：记录哪个数字出现过
            unsigned orv = 0;               // 这段区间的 OR
            for (int i = l; i <= r; i++) {
                orv |= a[i];                // 按位或：点亮新数的灯
                if (a[i] <= 100) vis[a[i]] = 1;
            }
            int mex = 0;                    // 从 0 开始，第一个没出现的就是 MEX
            while (vis[mex]) mex++;
            ans += orv | (ull)mex;          // 区间价值 = OR | MEX
        }
    printf("%llu\n", ans);
    return 0;
}
```

档 2　$O(n^2)$ 增量维护（点 1~6，30 分）

思路：固定左端点，右端点一格一格往右挪。挪的时候 OR 和 MEX 都**不用从头重算**，只要在上一轮的基础上「加一点点」： OR 直接把新数或进来；MEX 用一个桶 cnt 记次数，再用一根指针维护。 这根指

针**只会往右走、不会回头**（数一旦凑进来就不会消失），所以整个右端点扫一遍，指针最多也只走 n 步——这种「总共加起来不多」的分析方法叫**摊还**，平均到每一步就是 O(1)。

```
// 档 2：O(n^2) 增量维护（对应测试点 1~6，n <= 5000）
// 固定左端点，右端点一格一格往右挪，OR 和 MEX 都只要"加一点点"
#include <bits/stdc++.h>
using namespace std;
typedef unsigned long long ull;

int n;
unsigned *a;    // 数组按 n 动态分配，用多少开多少
int *cnt;       // cnt[v] = 当前区间里数字 v 出现了几次

int main() {
    scanf("%d", &n);
    a = new unsigned[n + 1];
    cnt = new int[n + 2];
    for (int i = 0; i < n; i++) scanf("%u", &a[i]);
    ull ans = 0;
    for (int l = 0; l < n; l++) {
        unsigned orv = 0;                   // 当前区间 [l, r] 的 OR
        int mex = 0;                        // 当前区间 [l, r] 的 MEX
        memset(cnt, 0, sizeof(int) * (n + 2));   // 一轮清零，比逐个减回去快
        for (int r = l; r < n; r++) {       // r 往右挪一格
            unsigned x = a[r];
            orv |= x;                       // OR 只会点亮新的灯
            if (x <= n) cnt[x]++;           // 比 n 大的数不影响 MEX
            while (cnt[mex]) mex++;         // mex 只升不降，摊还 O(1)
            ans += orv | (ull)mex;
        }
    }
    printf("%llu\n", ans);
    return 0;
}
```

两个细节帮你跑更快、更稳： ① 数组 a 和 cnt 用 new 按 n 动态分配——开小了会**越界**，数据一大程序就可能崩或死循环，所以「用多少开多少」最保险；② 换左端点时用 memset 一次性把桶清零，比逐个减回去快得多。这一档在 n ⩽ 5000 时几十毫秒就能跑完，n 到几万也能 hold 住。

4. 正解推导

$O(n^2)$ 到顶只能拿 30 分。想要满分，得换一种眼光——**盯着二进制位看**。

4.1 先看二进制：OR 就是点灯

把 0、1、2、3 的二进制排在一起（亮起的位标蓝）

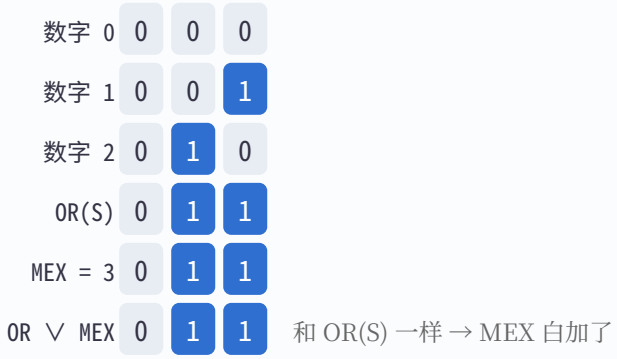


数字凑得越齐，点亮的灯越多；而且灯一旦点亮就不会再灭。

4.2 关键发现：MEX 的灯早就被 OR 点亮了

这是全题最关键的一步。如果 MEX = 3，说明 0、1、2 都在集合里（MEX 的定义：从 0 开始第一个缺席的数）。把这三个数 OR 起来看看：

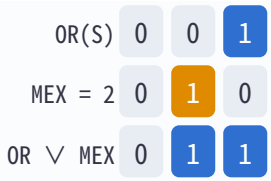
MEX = 3：OR 已经把 3 的灯全点亮，MEX 白加了



但如果 MEX = 2，集合里只有 0、1：

MEX = 2：OR 只有 001，MEX 的第 1 位是新的





第 1 位（橙色）是新的 → 比 OR 多了 2

把 m = 0 到 8 全部列出来，规律一目了然：

MEX = m	必有 0..m−1	OR(0..m−1)	m 的二进制	m 被覆盖?	f 比 OR 多
0	∅	0	0	是	0
1	0	0	1	否	1
2	0,1	1	10	否	2
3	0,1,2	3 (011)	11	是	0
4	0,1,2,3	3 (011)	100	否	4
5	0..4	7 (111)	101	是	0
6	0..5	7 (111)	110	是	0
7	0..6	7 (111)	111	是	0
8	0..7	7 (111)	1000	否	8

规律（全题灵魂）： 0..m−1 凑齐后会把低位全部点亮。m 若不是 2 的幂，它的每一位都落在这些低位里、被「吃掉」，于是 f = OR；**只有当 MEX = 1, 2, 4, 8, ...（2 的幂）时**，它比「低位全 1」多出一位，f = OR ∨ 2^k，这时才可能多算 2^k。

4.3 于是答案拆成两块

既然只有「MEX 是 2 的幂」的区间才可能比 OR 多一点，我们就把答案拆成两部分分别算：

$$\text{Ans} = \Sigma_{1\leqslant l\leqslant r\leqslant n}\ \text{OR}(S)\ +\ \Sigma_{k\geqslant 0}\ 2^k \cdot \text{cnt}_k$$

第一块：所有区间的 OR 之和（每个区间的保底价值）。第二块是**修正项**：对每个 k，数一数有多少个区间满足「MEX = 2^k 且 OR 的第 k 位还是 0」，这样的区间每个要多补 2^k：

$$\text{cnt}_k = \#\{\,[l,r] : \text{MEX}(S) = 2^k,\ \text{且 OR 的第 } k \text{ 位} = 0\,\}$$

k 只枚举到 20 就够了： 区间长度 ⩽ n，所以 MEX ⩽ n，进而 $2^k \leqslant n$。$n = 2\times10^6$ 时 k 最多到 20。这是本题能做快的关键——要处理的 k 非常少。

4.4 第一块：OR 之和 = 按位统计 + 数反面

第 b 位为 1 的区间，各贡献 2^b。但「含第 b 位」的区间不好直接数，那就**数反面**：把「第 b 位是 1 的元素」当作隔板，序列被切成一段段「第 b 位全是 0」的连续段。一段长 L 的全 0 段，里面有 $L(L+1)/2$ 个区间完全不含第 b 位。用总区间数减去它，就得到「含第 b 位」的区间数。

$$\Sigma_{l\leqslant r}\,\mathrm{OR}(S)=\Sigma_{b=0..29}\,2^b\cdot(N-N_b),\quad N=n(n+1)/2,\ N_b=\text{不含第 b 位的区间数}$$

拿样例 1 算一遍：区间总数 $N=6$。第 0 位的位图是 0,1,0 → 全 0 段长 1,1 → 不含区间 2 个 → 含 4 个 → 贡献 $4\times2^0=4$；第 1 位位图 0,0,1 → 段长 2 → 不含 3 个 → 含 3 个 → 贡献 $3\times2^1=6$；更高位全 0 → 贡献 0。**第一块 = 4 + 6 = 10**。

4.5 第二块：修正项的三步转化

这一步要解决的问题是：在「第 k 位全 0」的前提下，有多少个区间把 $0, 1, \cdots, 2^k-1$ 全部包含了？分三步想清楚：

1. **条件「OR 第 k 位为 0」把序列切成段：**遇到第 k 位是 1 的数，段就断开，只在段内数区间。
2. **段内「MEX = 2^k」等价于「凑齐 $0..2^k-1$」：**因为 2^k 自己的第 k 位是 1，它根本进不了段，所以只要「$0..2^k-1$ 全出现」就满足 $\mathrm{MEX}=2^k$，只剩这一个条件。
3. **固定右端点 i，数合法左端点：**维护每个值 v 在当前段里**最后**一次出现的位置 last[v]。要让 $0..2^k-1$ 都落在 [l, i] 里，l 必须不超过这些最后位置中最小的那个，记作 mn。于是合法左端点就是从段头 st 到 mn，共 **mn − st + 1** 个。

拿样例 1 执行一遍（k = 0 时只要凑齐 0；k = 1 时要凑齐 0 和 1）：

k	位置 i	数字	发生了什么	还缺几个值	mn	新增区间
0	1	0	记下 0 在位置 1	1→0	1	1（区间 [1,1]）
	2	1	第 0 位是 1，段断开重置	1	3	0
	3	2	不是要凑的数，不动	1	3	0
1	1	0	记下 0 在位置 1	2→1	1	0（还缺 1）
	2	1	记下 1 在位置 2	1→0	1	1（区间 [1,2]）
	3	2	第 1 位是 1，段断开重置	2	4	0

于是 $\mathrm{cnt}_0=1$、$\mathrm{cnt}_1=1$，修正项 $=1\times2^0+1\times2^1=3$。加上第一块的 10，答案 **13** √，和前面手算一致。

档 3　满分代码 O(n log n)

代码里开了三个辅助数组，先把它们的分工说清楚，读起来就不慌了：

- `last[v]`：数字 v 在当前段里**最后**一次出现的位置；
- `cnt[p]`：有多少个数字的最后位置**正好是 p**——靠它能快速找到「最小的最后位置」mn；
- `tl[]`：当前段碰过哪些数字——段断开时只需清这些，不用整个数组清零，省时间。

```
// 档 3：满分做法 O(n log n)（对应全部测试点，n <= 2×10^6）
// 三条结论：
//   1) 多数区间价值 = OR(S)；只有 MEX 是 2 的幂时才可能多一位
//   2) 所有区间的 OR 之和：一位一位地算"有哪些区间含这一位"
//   3) 修正项：数"位 k 全 0 的段内凑齐 0..2^k-1 的子区间"个数
#include <bits/stdc++.h>
using namespace std;
typedef unsigned long long ull;

int n;
unsigned *a;
int *last;   // last[v] = 数字 v 在当前段里最后一次出现的位置
int *cnt;    // cnt[p] = 有多少个数字的"最后位置"正好是 p
int *tl;     // tl[] = 当前段碰过哪些数字（段断开时只清这些，省时间）

inline int rd() {                       // 快读：n 有 200 万，比 cin/scanf 稳
    int x = 0, c = getchar();
    while (c != EOF && (c < '0' || c > '9')) c = getchar();
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c = getchar(); }
    return x;
}

int main() {
    n = rd();
    a = new unsigned[n + 1];
    last = new int[n + 2];
    cnt = new int[n + 2];
    tl = new int[n + 2];
    for (int i = 1; i <= n; i++) a[i] = rd();

    ull ans = 0;
    ull tot = (ull)n * (n + 1) / 2;     // 区间总数

    // ======== 第一步：所有区间的 OR 之和（按位统计）========
    for (int b = 0; b < 30; b++) {      // a_i < 2^30，看 0~29 位
        ull no = 0;                     // "这一位一个都没有"的区间数
        long long run = 0;              // 当前连续"这一位是 0"的长度
        for (int i = 1; i <= n; i++) {
            if (a[i] >> b & 1u) { no += (ull)run * (run + 1) / 2; run = 0; }
            else run++;
        }
        no += (ull)run * (run + 1) / 2; // 收尾
        ans += (tot - no) * (1ull << b);// 含这一位的区间数 × 2^b
    }

    // ======== 第二步：修正项，按 k 一个一个来 ========
    // 只需看 2^k <= n 的 k（MEX 不会超过 n，n=2×10^6 时 k 最多到 20）
```

```
    for (int k = 0; (1u << k) <= (unsigned)n; k++) {
        int need = 1 << k;              // 要凑齐 0, 1, ..., need-1
        for (int v = 0; v < need; v++) last[v] = 0;
        memset(cnt, 0, sizeof(int) * (n + 2));

        int st = 1;                     // 当前"第 k 位全 0 的段"从哪开始
        int rem = need;                 // 0..need-1 里还有几个没出现
        int mn = 1;                     // 所有 last 里最小的那个（单调不降）
        int tc = 0;                     // tl 里装了多少个
        ull c = 0;                      // 这个 k 数出的区间个数

        for (int i = 1; i <= n; i++) {
            unsigned x = a[i];
            if (x >> k & 1u) {          // 第 k 位是 1 → 段到此为止
                for (int j = 0; j < tc; j++) last[tl[j]] = 0;
                tc = 0; st = i + 1; rem = need; mn = st;
                continue;
            }
            if (x < (unsigned)need) {   // x 是要凑的数，记录位置
                int op = last[x];
                if (op >= st) cnt[op]--;// 旧位置作废，挪到新位置
                else rem--;             // 本段第一次见到 x
                last[x] = i; cnt[i]++; tl[tc++] = x;
                if (rem == 0) while (cnt[mn] == 0) mn++;   // 凑齐了，推进 mn
            }
            // 以 i 为右端点：左端点取 st..mn 都合法
            if (rem == 0) c += (ull)(mn - st + 1);
        }
        ans += c * (1ull << k);         // 每个这样的区间多算 2^k
    }
    printf("%llu\n", ans);
    return 0;
}
```

5. func 题常见坑

1. 取模：unsigned long long 自然溢出，输出 %llu；别用有符号 long long 溢出。
2. MEX ⩽ n：区间最多 n 个数，第一个缺席的数不会超过 n。大于 n 的值不用进桶，k 也只需到 20。
3. 数组别开小了：桶、序列都要按 n 开（或动态分配），开小了会越界，数据一大程序就崩或死循环。
4. 段断开要清 last：用 tl 只清碰过的值，别每轮 $O(2^k)$ 清空，会被卡。
5. mn 摊还：它只往右走，while 总共才推 n 次，不会慢。
6. 读入：$n = 2\times10^6$，用快读或关同步的 cin。
7. 位运算括号：(a[i] >> k) & 1u 的括号不能省。

6. 附录：怎么自己验证（对拍）

写完代码怎么知道对不对？用一个叫**对拍**的办法：再写一个最笨但绝对正确的暴力（两题的档 1 就是），然后写一个小脚本——随机生成很多小数据（n、m 都 ⩽ 8，值取小），把同一份输入同时喂给暴力和你的快速程序，比对输出，连跑几百组。全部一致，基本就稳了。

> **这套题解就是这么验证的：**file 题三档互相对拍数百组全一致；func 题三档对拍、并与一个独立的参考实现交叉比对数百组也全一致；两题的所有样例全部通过。

两题各一句话总结：

- **file**——每个元素的破坏力 = 比它大的核心个数，破坏力小的批次先归档，答案 = $m \cdot C_0 + \Sigma W \cdot (m - \text{序号} + 1)$。
- **func**——MEX 的灯早被 OR 点亮，只有 MEX 是 2 的幂时才多一位，答案 = 所有区间 OR 之和 + $\Sigma 2^k \times$（位 k 全 0 段内凑齐 $0..2^k-1$ 的区间数）。

> **给同学的最后建议：**这两道题的完美路径是一样的——先用暴力把题读懂、把样例算对，再观察「哪些是固定不变的、哪些才是顺序/位置真正影响的」，最后把发现写成公式。希望你以后遇到新题，也能这样一层一层往下拆。